

# Conexión a Base de datos desde Java usando JDBC

~~~~~ María Emilia Navarrete Ávila



# ¿Qué es JDBC?

1



JDBC es la API oficial de Java diseñada para proporcionar una conectividad universal con bases de datos relacionales, permitiendo la ejecución de sentencias SQL.

2



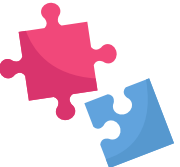
Provee un conjunto estandarizado de interfaces y clases que actúan como un puente comunicativo entre el lenguaje Java y el SGBD (Sistema Gestor de Base de Datos).

3



Su diseño permite que una misma aplicación pueda conectarse a diferentes motores como MySQL, PostgreSQL, Oracle o SQL Server sin alterar la lógica de negocio



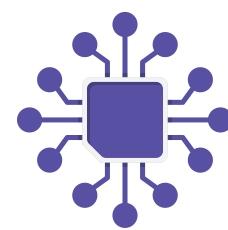


# El Desafío de la Persistencia de Datos

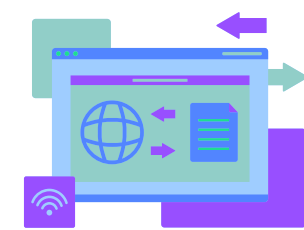


En el desarrollo de software empresarial, el desafío central es:

- Garantizar la persistencia de la información; sin una solución adecuada, los datos de los usuarios, inventarios o registros académicos serían volátiles y desaparecerían al finalizar el ciclo de vida de la ejecución del programa.



La limitación fundamental que resuelve JDBC es la falta de un protocolo nativo en Java para dialogar con los motores de almacenamiento, proporcionando una estructura estandarizada que permite enviar instrucciones SQL complejas y procesar resultados de manera eficiente



JDBC permite implementar operaciones CRUD (Create, Read, Update, Delete) de forma segura y consistente, garantizando que, independientemente de la complejidad del motor de base de datos, la aplicación mantenga su integridad operativa y capacidad de manipulación de datos.

# Componentes Principales de la Arquitectura

Es la clase gestora que carga los controladores (drivers) y establece la conexión inicial mediante la cadena de conexión (URL, usuario y password).



Aplicación Java



DriverManager



Representa la sesión física activa. A través de este objeto, controlamos la integridad de los datos mediante transacciones y gestionamos el ciclo de vida de la conexión.



Connection



Es el canal de transporte para las instrucciones SQL. Es la interfaz que permite al programa enviar comandos DML y DDL hacia el servidor.



Statement



Funciona como una tabla temporal cargada en memoria que contiene los resultados de una consulta SELECT, permitiendo al desarrollador navegar por ellos mediante un cursor.

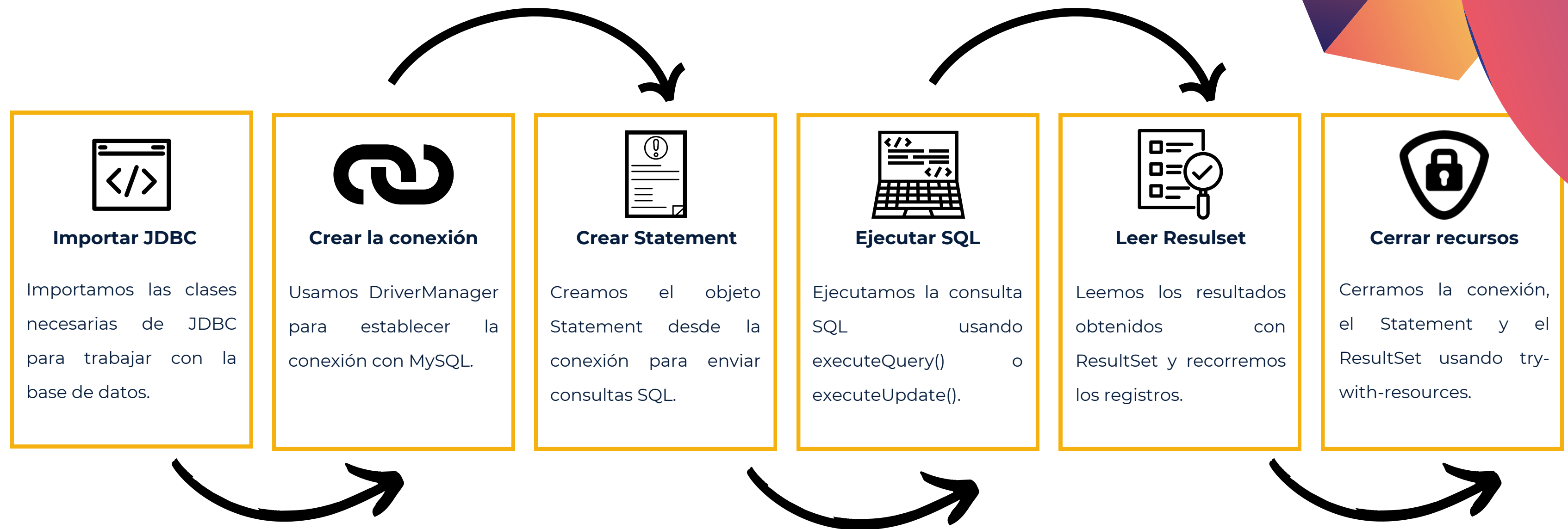


ResultSet





# El Flujo de Trabajo



La inmutabilidad de este proceso es fundamental para la estabilidad; cada componente depende estrictamente del éxito del anterior: una conexión mal establecida o un Statement no inicializado interrumpen instantáneamente la capacidad de persistencia, lo que resalta la importancia de la validación en cada etapa.

# Mejores Prácticas: Gestión de Recursos

- La implementación del bloque try-with-resources constituye la práctica más robusta y segura en Java moderno, ya que garantiza la gestión automática y el cierre de los objetos Connection, Statement y ResultSet.
- Esta estructura implementa la interfaz AutoCloseable, lo que asegura que, incluso ante la aparición de excepciones críticas, los recursos se cierran en el orden inverso al que fueron abiertos, evitando fugas de memoria.
- El uso de esta práctica previene errores de producción como Too many connections, que ocurren cuando el desarrollador olvida liberar los recursos, lo que podría llevar al bloqueo del servidor y a la inestabilidad del sistema.



# Aplicaciones en el Mundo Real



En el sector bancario, JDBC permite la implementación de transacciones bajo el paradigma ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), garantizando que las transferencias sean seguras y consistentes



En el ámbito del comercio electrónico, esta tecnología es el motor que gestiona el inventario, el procesamiento de pedidos en tiempo real y la sincronización constante de la información del cliente.



En plataformas académicas, JDBC facilita el manejo de expedientes complejos, el historial de calificaciones y la auditoría de registros de estudiantes, permitiendo realizar consultas analíticas complejas de manera eficiente.



# Ciclo de Operaciones: Conexión y Procesamiento

## ✓ *Conexión y Ejecución*

- La conexión se inicia mediante `DriverManager.getConnection()`, seguido de la creación del `Statement` o `PreparedStatement` para encapsular la lógica SQL.
- La ejecución de sentencias se bifurca en `executeQuery()` para consultas de lectura (SELECT) y `executeUpdate()` para operaciones de manipulación de datos (INSERT, UPDATE, DELETE).

## 🕒 *Procesamiento del ResultSet*

- Una vez ejecutada la consulta, el `ResultSet` actúa como un cursor posicionado antes de la primera fila, requiriendo el uso imperativo del bucle `while(resultado.next())` para recorrer los registros.
- La extracción de datos se realiza mediante nombres de columna para mejorar la legibilidad del código, mientras JDBC gestiona automáticamente la conversión entre tipos de datos SQL y primitivos de Java.



# EJEMPLO PRÁCTICO



CÓDICO JAVA



# *Conclusiones*

- DBC representa el pilar fundamental que todo desarrollador de software profesional debe dominar para comprender la persistencia de datos en el ecosistema Java.
- La adopción de buenas prácticas tales como la gestión automatizada de recursos, la prevención de inyección SQL y el manejo adecuado de excepciones marca la diferencia entre un código de estudiante y un sistema comercial estable y seguro.
- Finalmente, entender JDBC es el requisito técnico indispensable para avanzar hacia el dominio de frameworks de persistencia de alto nivel como JPA e Hibernate



**Gracias**